

# Resultados

December 7, 2025

```
[1]: import torch
import clip
from PIL import Image
import os
import torchvision.transforms as T
from tqdm import tqdm
from train_minitheia import TheiaStudent # Ajusta si tu archivo tiene otro
    nombre
import numpy as np
import matplotlib.pyplot as plt
import timm
import torch.nn as nn
import random
import warnings
warnings.filterwarnings('ignore')

from train_minitheia import TheiaStudent
```

```
/home/sergio/.local/lib/python3.12/site-packages/tqdm/auto.py:21: TqdmWarning:
IPProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
[2]: # -----
# CONFIG
# -----
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
MODEL_WEIGHTS = "runs/theia_mini/theia_mini_best.pt"
N_SAMPLES = 0

# Imágenes de prueba
IMAGE_PATHS = [
    "perro.jpg",
    "gato.jpg"
]

TEXT_PROMPTS = [
    "un perro corriendo en el césped",
```

```
"un gato mirando a la cámara"  
]
```

```
[3]: # -----  
# CARGAR MODELOS  
# -----  
print("Cargando CLIP teacher...")  
clip_model, clip_preprocess = clip.load("ViT-B/32", device=DEVICE, jit=False)  
  
print("Cargando Mini-Theia...")  
student = TheiaStudent().to(DEVICE)  
student.load_state_dict(torch.load(MODEL_WEIGHTS, map_location=DEVICE))  
student.eval()  
  
to_tensor = T.Compose([  
    T.Resize((224,224)),  
    T.ToTensor()  
)  
  
# -----  
# EXTRACCIÓN  
# -----  
def get_student_image_emb(path):  
    img = Image.open(path).convert("RGB")  
    tensor = to_tensor(img).unsqueeze(0).to(DEVICE)  
    with torch.no_grad():  
        with torch.cuda.amp.autocast():  
            clip_emb, dino_emb, fused = student(tensor) # fused = 192 dims  
    return clip_emb, fused  
  
def get_clip_teacher_image_emb(path):  
    img = Image.open(path).convert("RGB")  
    tensor = clip_preprocess(img).unsqueeze(0).to(DEVICE)  
    with torch.no_grad():  
        image_features = clip_model.encode_image(tensor)  
    return image_features  
  
def get_clip_text_emb(text):  
    with torch.no_grad():  
        text_tokens = clip.tokenize(text).to(DEVICE)  
        text_features = clip_model.encode_text(text_tokens)  
    return text_features  
  
# -----  
# SIMILITUD  
# -----
```

```
def cosine(a, b):
    return torch.nn.functional.cosine_similarity(a, b).item()
```

Cargando CLIP teacher...

Cargando Mini-Theia...

```
[4]: # ----- Retrieval Utility -----
def load_image(path):
    tf = T.Compose([
        T.Resize(256),
        T.CenterCrop(224),
        T.ToTensor(),
        T.Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]),
    ])
    return tf(Image.open(path).convert("RGB"))

def cosine_similarity(a, b):
    return (a @ b.T).cpu().numpy()

def show_results(query_path, top_paths):
    fig, axs = plt.subplots(1, len(top_paths) + 1, figsize=(16, 4))

    # Query
    axs[0].imshow(Image.open(query_path))
    axs[0].set_title("Query")
    axs[0].axis("off")

    # Top-K
    for i, p in enumerate(top_paths):
        axs[i+1].imshow(Image.open(p))
        axs[i+1].set_title(f"Top {i+1}")
        axs[i+1].axis("off")

    plt.tight_layout()
    plt.show()
```

```
[5]: dataset_dir = "test"
k = 5

# Load model
model = TheiaStudent()
model.load_state_dict(torch.load(MODEL_WEIGHTS, map_location=DEVICE))
model = model.to(DEVICE).eval()

def testDINO(image, model):
    # Load query
    query_img = load_image(image).unsqueeze(0).to(DEVICE)
```

```

# Compute query embedding
with torch.no_grad():
    _, dino_query, _ = model(query_img)

# Precompute gallery embeddings
gallery_paths = []
gallery_embeds = []

all_image_paths = []
for fname in os.listdir(dataset_dir):
    if fname.lower().endswith((".jpg", ".jpeg", ".png")):
        path = os.path.join(dataset_dir, fname)
        all_image_paths.append(path)
if N_SAMPLES == 0:
    selected_paths = all_image_paths
elif len(all_image_paths) > N_SAMPLES:
    selected_paths = random.sample(all_image_paths, N_SAMPLES) #
else:
    selected_paths = all_image_paths
print(f"Solo se encontraron {len(all_image_paths)} imágenes, procesando_
→todas.")

for path in tqdm(selected_paths):
    img = load_image(path).unsqueeze(0).to(DEVICE)

    with torch.no_grad():
        _, emb, _ = model(img)

    gallery_paths.append(path)
    gallery_embeds.append(emb.cpu().numpy()[0])

# Compute similarity
sims = gallery_embeds @ dino_query.cpu().numpy()[0]

# Top-K
top_idx = np.argsort(-sims)[:k]
top_paths = [gallery_paths[i] for i in top_idx]
# Show result
show_results(image, top_paths)

```

```

[6]: # -----
# EVALUACIÓN
# -----
print("\n== Evaluación CLIP texto  imagen ==")

for text in TEXT_PROMPTS:

```

```

print(f"\n Texto: {text}")

teacher_text = get_clip_text_emb(text)

best_img = None
best_teacher_score = -1
best_student_score = -1

for img_path in IMAGE_PATHS:
    teacher_img = get_clip_teacher_image_emb(img_path)
    student_img, fused = get_student_image_emb(img_path)

    s_t = cosine(student_img, teacher_text)
    t_t = cosine(teacher_img, teacher_text)

    print(f" Imagen: {img_path}")
    print(f" Teacher CLIP score: {t_t:.4f}")
    print(f" Student Mini-Theia score: {s_t:.4f}")

    if s_t > best_student_score:
        best_student_score = s_t
        best_img = img_path

print(f" Mini-Theia considera más similar: **{best_img}**")

```

=== Evaluación CLIP texto imagen ===

```

Texto: un perro corriendo en el césped
Imagen: perro.jpg
  Teacher CLIP score: 0.2563
  Student Mini-Theia score: 0.2507
Imagen: gato.jpg
  Teacher CLIP score: 0.1710
  Student Mini-Theia score: 0.2049
Mini-Theia considera más similar: **perro.jpg**

Texto: un gato mirando a la cámara
Imagen: perro.jpg
  Teacher CLIP score: 0.1799
  Student Mini-Theia score: 0.2294
Imagen: gato.jpg
  Teacher CLIP score: 0.2520
  Student Mini-Theia score: 0.2343
Mini-Theia considera más similar: **gato.jpg**

```

```
[7]: testDINO("query.jpg", model)
testDINO("perro.jpg", model)
testDINO("gato.jpg", model)
```

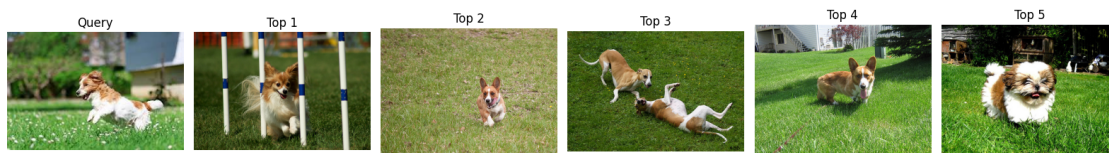
100%|

40152/40152 [04:08<00:00, 161.36it/s]



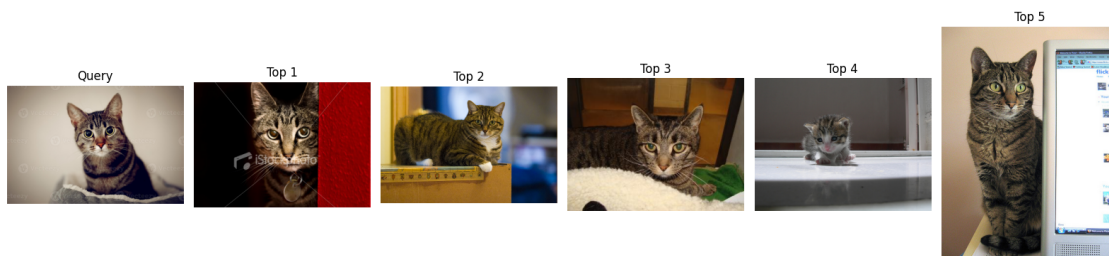
100%|

40152/40152 [04:11<00:00, 159.35it/s]



100%|

40152/40152 [04:11<00:00, 159.92it/s]



```
[8]: from theia_triple import TheiaStudent
MODEL_WEIGHTS = "runs/theia_triple/theia_triple_best.pt"
```

```
[9]: # -----
# CARGAR MODELOS
# -----
print("Cargando CLIP teacher...")
clip_model, clip_preprocess = clip.load("ViT-B/32", device=DEVICE, jit=False)

print("Cargando Mini-Theia (CLIP+DINO+FastSAM)...")
student = TheiaStudent(
    backbone_name='vit_tiny_patch16_224',
    proj_clip=512,
    proj_dino=384,
    proj_fastsam=256
).to(DEVICE)

student.load_state_dict(torch.load(MODEL_WEIGHTS, map_location=DEVICE))
student.eval()

# Transformaciones
student_transform = T.Compose([
    T.Resize(int(224 * 1.14)),
    T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

# -----
# EXTRACCIÓN
# -----
def get_student_clip_emb(path):
    img = Image.open(path).convert("RGB")
    tensor = student_transform(img).unsqueeze(0).to(DEVICE)
    with torch.no_grad():
        clip_emb, _, _, _ = student(tensor) # Solo usamos salida CLIP
    return clip_emb

def get_clip_teacher_image_emb(path):
    img = Image.open(path).convert("RGB")
    tensor = clip_preprocess(img).unsqueeze(0).to(DEVICE)
    with torch.no_grad():
        image_features = clip_model.encode_image(tensor)
        image_features = image_features / image_features.norm(dim=-1,
↪keepdim=True)
    return image_features

def get_clip_text_emb(text):
    with torch.no_grad():
        text_tokens = clip.tokenize(text).to(DEVICE)
```

```

        text_features = clip_model.encode_text(text_tokens)
        text_features = text_features / text_features.norm(dim=-1, keepdim=True)
    return text_features

# -----
# SIMILITUD
# -----
def cosine(a, b):
    return torch.nn.functional.cosine_similarity(a, b).item()

```

Cargando CLIP teacher...

Cargando Mini-Theia (CLIP+DINO+FastSAM)...

```

[10]: # -----
# EVALUACIÓN
# -----
print("\n=== Evaluación del componente CLIP (texto  imagen) ===")
print(f"Comparando: {len(IMAGE_PATHS)} imágenes vs {len(TEXT_PROMPTS)}\n↳ textos\n")

for text in TEXT_PROMPTS:
    print(f"  Texto: '{text}'")

    # Embedding del texto (teacher CLIP)
    teacher_text = get_clip_text_emb(text)

    best_img = None
    best_teacher_score = -1
    best_student_score = -1

    for img_path in IMAGE_PATHS:
        # Embeddings de imagen
        teacher_img = get_clip_teacher_image_emb(img_path)
        student_clip_emb = get_student_clip_emb(img_path)

        # Calcular similitudes
        s_t = cosine(student_clip_emb, teacher_text)
        t_t = cosine(teacher_img, teacher_text)

        print(f"  Imagen: {img_path}")
        print(f"    Teacher CLIP score: {t_t:.4f}")
        print(f"    Student (CLIP head) score: {s_t:.4f}")
        print(f"    Diferencia: {abs(t_t - s_t):.4f}")

        if s_t > best_student_score:
            best_student_score = s_t
            best_img = img_path

```

```

        if t_t > best_teacher_score:
            best_teacher_score = t_t

    print(f" El estudiante (componente CLIP) considera más similar:␣
↪**{best_img}** (score: {best_student_score:.4f})")
    print(f" El teacher CLIP considera más similar: **{p for p in IMAGE_PATHS␣
↪if p != best_img}[0] if best_teacher_score > best_student_score else␣
↪best_img}** (score: {best_teacher_score:.4f})")
    print("-" * 50)

```

=== Evaluación del componente CLIP (texto    imagen) ===  
Comparando: 2 imágenes vs 2 textos

```

Texto: 'un perro corriendo en el césped'
Imagen: perro.jpg
Teacher CLIP score: 0.2563
Student (CLIP head) score: 0.2630
Diferencia: 0.0066
Imagen: gato.jpg
Teacher CLIP score: 0.1710
Student (CLIP head) score: 0.1800
Diferencia: 0.0090
El estudiante (componente CLIP) considera más similar: **perro.jpg** (score:
0.2630)
El teacher CLIP considera más similar: **perro.jpg** (score: 0.2563)
-----
Texto: 'un gato mirando a la cámara'
Imagen: perro.jpg
Teacher CLIP score: 0.1801
Student (CLIP head) score: 0.2090
Diferencia: 0.0289
Imagen: gato.jpg
Teacher CLIP score: 0.2520
Student (CLIP head) score: 0.2459
Diferencia: 0.0061
El estudiante (componente CLIP) considera más similar: **gato.jpg** (score:
0.2459)
El teacher CLIP considera más similar: **perro.jpg** (score: 0.2520)
-----

```

```

[11]: # -----
# Funciones de utilidad
# -----
def load_image(path, size=224):
    """Carga y transforma una imagen para el estudiante"""
    tf = T.Compose([

```

```

        T.Resize(int(size * 1.14)),
        T.CenterCrop(size),
        T.ToTensor(),
        T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ])
    return tf(Image.open(path).convert("RGB"))

def cosine_similarity(a, b):
    """Calcula similitud coseno entre vectores"""
    return (a @ b.T).cpu().numpy()

def show_results(query_path, top_paths, scores, title="Resultados DINO"):
    """Muestra los resultados visualmente"""
    n_results = len(top_paths)
    fig, axes = plt.subplots(1, n_results + 1, figsize=(4*(n_results + 1), 4))

    # Query
    axes[0].imshow(Image.open(query_path))
    axes[0].set_title("Query", fontsize=12, fontweight='bold')
    axes[0].axis("off")

    # Top-K resultados
    for i, (path, score) in enumerate(zip(top_paths, scores)):
        axes[i+1].imshow(Image.open(path))
        axes[i+1].set_title(f"Top {i+1}\nscore: {score:.3f}", fontsize=10)
        axes[i+1].axis("off")

    plt.suptitle(title, fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

```

```

[12]: def compute_similarity_matrix(embeddings):
    """Calcula matriz de similitud entre todos los embeddings"""
    n = len(embeddings)
    similarity_matrix = np.zeros((n, n))

    for i in range(n):
        for j in range(n):
            similarity_matrix[i, j] = np.dot(embeddings[i], embeddings[j])

    return similarity_matrix

def visualize_similarity_matrix(paths, similarity_matrix, title="Matriz de Similitud FastSAM"):
    """Visualiza la matriz de similitud"""
    n = len(paths)

```

```

fig, ax = plt.subplots(figsize=(max(8, n//2), max(6, n//3)))

im = ax.imshow(similarity_matrix, cmap='viridis')

# Configurar etiquetas
short_names = [os.path.basename(p)[:10] for p in paths]
ax.set_xticks(np.arange(n))
ax.set_yticks(np.arange(n))
ax.set_xticklabels(short_names, rotation=45, ha='right')
ax.set_yticklabels(short_names)

# Añadir valores en las celdas
for i in range(n):
    for j in range(n):
        text = ax.text(j, i, f'{similarity_matrix[i, j]:.2f}',
                        ha="center", va="center",
                        color="w" if similarity_matrix[i, j] < 0.5 else "k",
                        fontsize=8)

ax.set_title(title, fontsize=14, fontweight='bold')
plt.colorbar(im, ax=ax, label='Similitud Coseno')
plt.tight_layout()
plt.show()

def show_results(query_path, top_paths, scores, title="Resultados FastSAM"):
    """Muestra los resultados visualmente"""
    n_results = len(top_paths)
    fig, axes = plt.subplots(1, n_results + 1, figsize=(4*(n_results + 1), 4))

    # Query
    axes[0].imshow(Image.open(query_path))
    axes[0].set_title("Query", fontsize=12, fontweight='bold')
    axes[0].axis("off")

    # Top-K resultados
    for i, (path, score) in enumerate(zip(top_paths, scores)):
        axes[i+1].imshow(Image.open(path))
        axes[i+1].set_title(f"Top {i+1}\nscore: {score:.3f}", fontsize=10)
        axes[i+1].axis("off")

    plt.suptitle(title, fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

```

```

[13]: # -----
# Función principal
# -----

```

```

print(f"Usando dispositivo: {DEVICE}")

gallery = "test"
k = 5

if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")

# 1. Cargar modelo
print(f"\n Cargando modelo desde: {MODEL_WEIGHTS}")
student = TheiaStudent(
    backbone_name='vit_tiny_patch16_224',
    proj_clip=512,
    proj_dino=384,
    proj_fastsam=256
)

try:
    student.load_state_dict(torch.load(MODEL_WEIGHTS, map_location=DEVICE))
    print(" Modelo cargado exitosamente")
except Exception as e:
    print(f" Error cargando modelo: {e}")

student = student.to(DEVICE).eval()

def testDINO(query, student):
    # 2. Cargar y procesar imagen de consulta
    print(f"\n Procesando imagen de consulta: {query}")
    if not os.path.exists(query):
        print(f" No se encuentra la imagen de consulta: {query}")
        return

    query_img = load_image(query).unsqueeze(0).to(DEVICE)

    # Extraer embedding DINO de la consulta
    with torch.no_grad():
        _, dino_query, _, _ = student(query_img)

    print(f" Embedding DINO extraído (dimensión: {dino_query.shape[1]})")

    # 3. Procesar galería de imágenes
    print(f"\n Explorando galería: {gallery}")
    if not os.path.exists(gallery):
        print(f" No se encuentra el directorio de galería: {gallery}")
        return

    gallery_paths = []

```

```

gallery_embeds = []

all_image_paths = []
for fname in os.listdir(gallery):
    if fname.lower().endswith((".jpg", ".jpeg", ".png")):
        path = os.path.join(gallery, fname)
        all_image_paths.append(path)

if N_SAMPLES == 0:
    selected_paths = all_image_paths
elif len(all_image_paths) > N_SAMPLES:
    selected_paths = random.sample(all_image_paths, N_SAMPLES) #
else:
    selected_paths = all_image_paths
print(f"Solo se encontraron {len(all_image_paths)} imágenes, procesando_
todas.")

for path in tqdm(selected_paths):
    img = load_image(path).unsqueeze(0).to(DEVICE)

    with torch.no_grad():
        _, emb, _, _ = student(img)

    gallery_paths.append(path)
    gallery_embeds.append(emb.cpu().numpy()[0])

gallery_embeds = np.stack(gallery_embeds)
print(f" {len(gallery_embeds)} embeddings extraídos exitosamente")

# 4. Calcular similitudes
print("\n Calculando similitudes coseno...")
sims = gallery_embeds @ dino_query.cpu().numpy()[0]

# 5. Obtener top-K resultados
top_idx = np.argsort(-sims)[:k]
top_paths = [gallery_paths[i] for i in top_idx]
top_scores = sims[top_idx]

# 6. Mostrar resultados
print(f"\n{'='*60}")
print(" TOP-K RESULTADOS (Componente DINO)")
print(f"{'='*60}")

print(f"Consulta: {query}")
print(f"Galería: {gallery} ({len(gallery_paths)} imágenes)")
print(f"K = {k}\n")

```

```

print("Rank | Similaridad | Imagen")
print("-" * 60)

for i, (path, score) in enumerate(zip(top_paths, top_scores)):
    img_name = os.path.basename(path)
    print(f"{i+1:4d} | {score:10.4f} | {img_name}")

# 7. Mostrar visualización
print(f"\n Mostrando visualización de resultados...")
show_results(query, top_paths, top_scores,
              title=f"Retrieval DINO - Top {k} resultados")

def testSAM(query, student):
    # 2. Cargar y procesar imagen de consulta
    print(f"\n Procesando imagen de consulta: {query}")
    if not os.path.exists(query):
        print(f" No se encuentra la imagen de consulta: {query}")
        return

    query_img = load_image(query).unsqueeze(0).to(DEVICE)

    # Extraer embedding FastSAM de la consulta
    with torch.no_grad():
        _, _, fastsam_query, _ = student(query_img)

    print(f" Embedding FastSAM extraído (dimensión: {fastsam_query.shape[1]})")

    # 3. Procesar galería de imágenes
    print(f"\n Explorando galería: {gallery}")
    if not os.path.exists(gallery):
        print(f" No se encuentra el directorio de galería: {gallery}")
        return

    gallery_paths = []
    gallery_embeds = []

    # Listar imágenes válidas
    valid_extensions = ('.jpg', '.jpeg', '.png', '.bmp', '.tiff')
    image_files = [f for f in os.listdir(gallery)
                    if f.lower().endswith(valid_extensions)]

    if len(image_files) == 0:
        print(" No se encontraron imágenes en la galería")
        return

    print(f"Encontradas {len(image_files)} imágenes en la galería")

```

```

# Extraer embeddings de todas las imágenes
for fname in tqdm(image_files, desc="Extrayendo embeddings FastSAM"):
    path = os.path.join(gallery, fname)

    try:
        img = load_image(path).unsqueeze(0).to(DEVICE)

        with torch.no_grad():
            _, _, emb, _ = student(img)

        gallery_paths.append(path)
        gallery_embeddings.append(emb.cpu().numpy()[0])
    except Exception as e:
        print(f"Error procesando {fname}: {e}")
        continue

if len(gallery_embeddings) == 0:
    print("No se pudieron extraer embeddings de la galería")
    return

gallery_embeddings = np.stack(gallery_embeddings)
print(f"{len(gallery_embeddings)} embeddings extraídos exitosamente")

# 4. Calcular similitudes
print("\n Calculando similitudes coseno...")
sims = gallery_embeddings @ fastsam_query.cpu().numpy()[0]

# 5. Obtener top-K resultados
top_idx = np.argsort(-sims)[:k]
top_paths = [gallery_paths[i] for i in top_idx]
top_scores = sims[top_idx]

# 6. Mostrar resultados
print(f"\n{'='*60}")
print("TOP-K RESULTADOS (Componente FastSAM)")
print(f"{'='*60}")

print(f"Consulta: {query}")
print(f"Galería: {gallery} ({len(gallery_paths)} imágenes)")
print(f"K = {k}\n")

print("Rank | Similitud | Imagen")
print("-" * 60)

for i, (path, score) in enumerate(zip(top_paths, top_scores)):
    img_name = os.path.basename(path)
    print(f"{i+1:4d} | {score:10.4f} | {img_name}")

```

```
# 7. Mostrar visualización
print(f"\n Mostrando visualización de resultados...")
show_results(query, top_paths, top_scores,
              title=f"Retrieval FastSAM - Top {k} resultados")
```

Usando dispositivo: cuda  
GPU: NVIDIA GeForce RTX 4070 SUPER

Cargando modelo desde: runs/theia\_triple/theia\_triple\_best.pt  
Modelo cargado exitosamente

```
[14]: testDINO("query.jpg",student)
testDINO("perro.jpg",student)
testDINO("gato.jpg",student)
```

Procesando imagen de consulta: query.jpg  
Embedding DINO extraído (dimensión: 384)

Explorando galería: test

100%|

|

40152/40152 [04:21<00:00, 153.58it/s]

40152 embeddings extraídos exitosamente

Calculando similitudes coseno...

```
=====
TOP-K RESULTADOS (Componente DINO)
=====
Consulta: query.jpg
Galería: test (40152 imágenes)
K = 5
```

Rank | Similaridad | Imagen

```
-----
1 | 0.7876 | ILSVRC2012_test_00019186.JPEG
2 | 0.7606 | ILSVRC2012_test_00060245.JPEG
3 | 0.7330 | ILSVRC2012_test_00039779.JPEG
4 | 0.7228 | ILSVRC2012_test_00010478.JPEG
5 | 0.7179 | ILSVRC2012_test_00015529.JPEG
```

Mostrando visualización de resultados...



Procesando imagen de consulta: perro.jpg  
 Embedding DINO extraído (dimensión: 384)

Explorando galería: test

100%|

40152/40152 [04:26<00:00, 150.87it/s]

40152 embeddings extraídos exitosamente

Calculando similitudes coseno...

=====

TOP-K RESULTADOS (Componente DINO)

=====

Consulta: perro.jpg  
 Galería: test (40152 imágenes)  
 K = 5

Rank | Similitud | Imagen

-----

|   |        |                               |
|---|--------|-------------------------------|
| 1 | 0.8342 | ILSVRC2012_test_00068579.JPEG |
| 2 | 0.8232 | ILSVRC2012_test_00066703.JPEG |
| 3 | 0.8174 | ILSVRC2012_test_00033371.JPEG |
| 4 | 0.8165 | ILSVRC2012_test_00085304.JPEG |
| 5 | 0.8103 | ILSVRC2012_test_00008419.JPEG |

Mostrando visualización de resultados...



Procesando imagen de consulta: gato.jpg  
Embedding DINO extraído (dimensión: 384)

Explorando galería: test

100%|

40152/40152 [04:28<00:00, 149.80it/s]

40152 embeddings extraídos exitosamente

Calculando similitudes coseno...

=====

TOP-K RESULTADOS (Componente DINO)

=====

Consulta: gato.jpg

Galería: test (40152 imágenes)

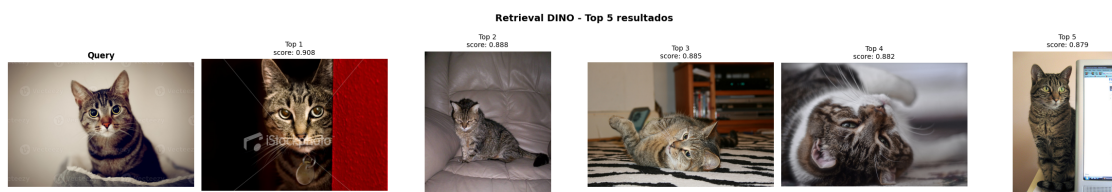
K = 5

Rank | Similaridad | Imagen

-----

|   |        |                               |
|---|--------|-------------------------------|
| 1 | 0.9083 | ILSVRC2012_test_00015726.JPEG |
| 2 | 0.8881 | ILSVRC2012_test_00071094.JPEG |
| 3 | 0.8850 | ILSVRC2012_test_00052136.JPEG |
| 4 | 0.8823 | ILSVRC2013_test_00005284.JPEG |
| 5 | 0.8790 | ILSVRC2012_test_00029669.JPEG |

Mostrando visualización de resultados...



```
[15]: testSAM("query.jpg",student)
      testSAM("perro.jpg",student)
      testSAM("gato.jpg",student)
```

Procesando imagen de consulta: query.jpg  
Embedding FastSAM extraído (dimensión: 256)

Explorando galería: test

Encontradas 40152 imágenes en la galería

Extrayendo embeddings FastSAM: 100%|

40152/40152 [04:27<00:00, 150.31it/s]

40152 embeddings extraídos exitosamente

Calculando similitudes coseno...

=====

TOP-K RESULTADOS (Componente FastSAM)

=====

Consulta: query.jpg

Galería: test (40152 imágenes)

K = 5

Rank | Similitud | Imagen

-----

|   |        |                               |
|---|--------|-------------------------------|
| 1 | 0.8431 | ILSVRC2012_test_00010478.JPEG |
| 2 | 0.8021 | ILSVRC2013_test_00007685.JPEG |
| 3 | 0.7992 | ILSVRC2012_test_00022298.JPEG |
| 4 | 0.7644 | ILSVRC2012_test_00019186.JPEG |
| 5 | 0.7600 | ILSVRC2012_test_00060245.JPEG |

Mostrando visualización de resultados...



Procesando imagen de consulta: perro.jpg

Embedding FastSAM extraído (dimensión: 256)

Explorando galería: test

Encontradas 40152 imágenes en la galería

Extrayendo embeddings FastSAM: 100%|

40152/40152 [04:27<00:00, 149.93it/s]

40152 embeddings extraídos exitosamente

Calculando similitudes coseno...

=====

## TOP-K RESULTADOS (Componente FastSAM)

=====

Consulta: perro.jpg  
Galería: test (40152 imágenes)  
K = 5

| Rank | Similaridad | Imagen |
|------|-------------|--------|
|------|-------------|--------|

|   |        |                               |
|---|--------|-------------------------------|
| 1 | 0.8266 | ILSVRC2012_test_00079756.JPEG |
| 2 | 0.8253 | ILSVRC2012_test_00083533.JPEG |
| 3 | 0.8220 | ILSVRC2012_test_00068579.JPEG |
| 4 | 0.8199 | ILSVRC2012_test_00001270.JPEG |
| 5 | 0.8153 | ILSVRC2012_test_00042179.JPEG |

Mostrando visualización de resultados...



Procesando imagen de consulta: gato.jpg  
Embedding FastSAM extraído (dimensión: 256)

Explorando galería: test  
Encontradas 40152 imágenes en la galería

Extrayendo embeddings FastSAM: 100%|

40152/40152 [04:29<00:00, 149.07it/s]

40152 embeddings extraídos exitosamente

Calculando similitudes coseno...

## TOP-K RESULTADOS (Componente FastSAM)

=====

Consulta: gato.jpg  
Galería: test (40152 imágenes)  
K = 5

| Rank | Similaridad | Imagen |
|------|-------------|--------|
|------|-------------|--------|

|   |        |                               |
|---|--------|-------------------------------|
| 1 | 0.7068 | ILSVRC2012_test_00080571.JPEG |
| 2 | 0.6962 | ILSVRC2012_test_00080355.JPEG |
| 3 | 0.6941 | ILSVRC2012_test_00044055.JPEG |
| 4 | 0.6864 | ILSVRC2012_test_00006867.JPEG |
| 5 | 0.6840 | ILSVRC2012_test_00008830.JPEG |

Mostrando visualización de resultados...

